

114學年 微算機原理與實習

超音波感測器 HC-SR04

距離偵測 × LED 警示 × 蜂鳴器警報

Arduino Mega 2560

ATmega2560

HC-SR04

學習目標



硬體知識

- HC-SR04 超音波感測器工作原理
- LED 接線與限流電阻計算
- 無源蜂鳴器 vs 有源蜂鳴器差異
- Arduino Mega 2560 腳位配置



程式概念

- 微電腦五大單元與程式對應
- ATmega2560 暫存器操作原理
- pulseIn() 計時機制
- tone() / noTone() PWM 應用
- 函式設計與模組化

微電腦五大單元

本實習各元件對應的五大單元分工

① 輸入單元 (Input Unit)

硬體：HC-SR04 ECHO 腳 (D11) → 接收超音波反射回波

暫存器：PINx — 讀取腳位電位狀態 (唯讀)

② 輸出單元 (Output Unit)

硬體：TRIG (D10)、LED (D6)、蜂鳴器 (D8)

暫存器：PORTx — 設定輸出電位高/低

③ 記憶單元 (Memory Unit)

Flash ROM 256KB：儲存程式碼 | SRAM 8KB：儲存執行時變數 (dist, duration) | EEPROM 4KB：掉電保存區

④ 算術邏輯單元 (ALU)

執行距離換算： $\text{duration} \times 0.034 \div 2$ | 條件判斷： $\text{dist} > 0 \ \&\& \ \text{dist} < 20$
| 迴圈計數： $i++$

⑤ 控制單元 (Control Unit)

程式計數器 (PC) 依序執行指令 | 遇 if/for 決定跳轉位址 | 呼叫函式時將返回位址壓入堆疊

ATmega2560 I/O 暫存器

Arduino 每個腳位都對應三個 8-bit 暫存器：

暫存器	全名	功能	Arduino 函式	範例
DDRx	Data Direction Register	設定腳位方向 0=INPUT, 1=OUTPUT	pinMode()	DDRB = (1 << PB4)
PORTx	Port Output Register	設定輸出電位 0=LOW(0V), 1=HIGH(5V)	digitalWrite()	PORTB = (1 << PB4)
PINx	Port Input Register	讀取輸入電位 (唯讀)	digitalRead()	if(PINB & (1 << PB4))

💡 Arduino 的 `pinMode()` / `digitalWrite()` / `digitalRead()` 是對 `DDRx` / `PORTx` / `PINx` 暫存器操作的高階封裝，底層都是在操作這三個暫存器的特定位元。

HC-SR04 工作原理

時序流程

1. TRIG 腳輸入 $\geq 10\mu s$ 高電位脈衝
2. 感測器發射 8 個 40kHz 超音波
3. 超音波碰到障礙物後反射
4. ECHO 腳輸出高電位 (時間 = 來回飛行時間)
5. Arduino 用 `pulseIn()` 測量 ECHO 高電位時間

腳位說明

HC-SR04 腳	接 Mega 2560
VCC	5V
GND	GND
TRIG	D10 (輸出)
ECHO	D11 (輸入)

 有效偵測範圍：2cm ~ 400cm
精確度：±3mm
偵測角度：15°以內效果最佳

距離換算公式 (ALU 執行)

聲速 $\approx 340 \text{ m/s} = 0.034 \text{ cm}/\mu s$

$\text{distance} = \text{duration} \times 0.034 \div 2$

÷2 原因：超音波「去+回」共走兩倍距離

pulseIn() 底層計時機制

函式用法

```
// 等待 ECHO 腳出現高電位  
// 並回傳高電位持續時間（微秒）  
long duration =  
pulseIn(ECHO_PIN, HIGH);
```

底層暫存器操作

- 使用 Timer1 (16-bit 計時器)
- 設定 TCCR1B 控制暫存器啟動計時
- 讀取 TCNT1 計數暫存器獲得時間
- ATmega2560 @ 16MHz → 解析度 1μs

計時流程

1. 等待 ECHO 腳從 LOW 變 HIGH
2. 記錄當前 TCNT1 值 (開始計時)
3. 持續檢查 ECHO 是否變 LOW
4. 記錄當前 TCNT1 值 (停止計時)
5. 回傳 (結束 - 開始) × 時脈週期

⌚ 預設逾時：1秒
(1,000,000μs)
若距離超過 400cm 或無障礙物，
pulseIn() 會逾時回傳 0

無源蜂鳴器與 tone()

有源 vs 無源蜂鳴器

	有源蜂鳴器	無源蜂鳴器
驅動方式	HIGH/LOW	PWM 方波
音調	固定	可調頻率
Arduino 函式	digitalWrite()	tone()
辨識方法	有「+」標示	無標示

tone() 底層 : Timer2 PWM

- 設定 TCCR2A/B 控制暫存器
- 設定 OCR2A 比較暫存器決定頻率
- Timer2 到達 OCR2A 值時翻轉輸出電位
- 蜂鳴器振膜依 PWM 頻率振動發聲

```
// 本程式使用方式：  
tone(BUZZER_PIN,  
1000); // 1000Hz 發聲  
  
noTone(BUZZER_PIN);  
// 停止發聲
```

程式架構說明

setup()

初始化，上電後
執行一次

- pinMode() 設定 DDRx
- Serial.begin() 設定 UART

getDistance()

距離測量函式

- 輸出 TRIG 脈衝
- pulseIn() 計時
- ALU 換算距離
- 回傳公分值

alert()

警報輸出
函式

- LED 閃爍 10 次
- tone() 同步發聲
- for 迴圈控制次數

loop() 主迴圈流程

上電 → setup() → 無限迴圈：

getDistance() → 印出距離 → 距離 < 20cm? → alert() →
delay(200ms) → 重複

關鍵程式碼解析

```
// ① 輸出觸發脈衝（輸出單元 → PORTx 暫存器）
digitalWrite(TRIG_PIN, LOW);    delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH);   delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

// ② 計時回波（輸入單元 → PINx 暫存器 + Timer1 計時暫存器）
long duration = pulseIn(ECHO_PIN, HIGH);

// ③ 距離換算（ALU 執行運算）
long distance = duration * 0.034 / 2;

// ④ 條件判斷（控制單元 → 決定跳轉）
if (dist > 0 && dist < ALERT_DIST) {
    alert();    // 控制單元：壓堆疊 → 跳轉 → 返回
}

// ⑤ 警報輸出（輸出單元 → PORTx + Timer2 PWM）
digitalWrite(LED_PIN, HIGH);
tone(BUZZER_PIN, 1000);
```

課堂練習題

基礎題

1. 將觸發距離改為 30cm
2. 將閃爍次數改為 5 次
3. 將蜂鳴器頻率改為 2000Hz
(音調更高)
4. 在序列埠監控視窗觀察距離輸出

觀念題

1. 為何距離公式要 $\div 2$?
2. 有源蜂鳴器和無源蜂鳴器哪個能變換音調 ?
3. DDRx / PORTx / PINx 暫存器各負責什麼 ?

進階題

1. 加入三色 LED，依距離遠近顯示不同顏色
 - 綠 : $> 50\text{cm}$
 - 黃 : $20\sim 50\text{cm}$
 - 紅 : $< 20\text{cm}$
2. 修改 alert() 讓蜂鳴器以不同頻率交替鳴叫 (救護車效果)
3. 用序列埠接收命令動態修改觸發距離

範例二

超音波偵測行人紅綠燈

綠燈結束偵測行人 → 自動延長通行時間

millis() 非阻塞

三色 LED

狀態控制

行人紅綠燈：功能設計

設計概念

- 模擬真實行人號誌運作
- 綠燈倒數結束時，超音波掃描是否仍有行人
- 偵測到行人 → 自動延長綠燈 3 秒
- 最多延長 2 次，防止號誌卡住
- 延長結束後正常進入黃燈 → 紅燈循環

接線說明

元件	Mega 2560
HC-SR04 TRIG	D10
HC-SR04 ECHO	D11
 綠燈 LED	D4 (串 220Ω)
 黃燈 LED	D5 (串 220Ω)
 紅燈 LED	D6 (串 220Ω)

號誌時序

燈號	時間	說明
 綠燈	5 秒	行人通行
 延長	+3 秒/次	有人則延長，最多2次
 黃燈	2 秒	準備停止
 紅燈	5 秒	禁止通行

 感應距離設定 150cm
模擬斑馬線寬度範圍

millis() 非阻塞計時

delay() 的問題

```
// delay() 阻塞 CPU，等待  
期間  
// 無法做任何其他事（包含讀  
按鈕）  
delay(5000); // CPU 卡在  
這裡 5 秒
```

millis() 非阻塞寫法

```
// millis(): Timer0 計數  
暫存器  
// 回傳上電後累計毫秒數  
unsigned long start =  
millis();  
while (millis() - start  
< 5000) {  
    // 等待期間可做其他事  
    checkSensor();  
    delay(50);  
}
```

millis() 底層原理

- Timer0 每 1ms 觸發中斷
- 中斷服務程式 (ISR) 將計數變數 +1
- millis() 讀取此計數變數回傳
- 計數器為 unsigned long (32位元)
- 約 49.7 天後溢位歸零

⚡ 本範例使用 millis() 等待綠燈時間，期間可同時做其他邏輯判斷，比 delay() 更接近真實系統設計。

行人紅綠燈：關鍵程式碼

```
// ① 綠燈開始，記錄起始時間 (ALU + 記憶單元)
setLight(true, false, false);
unsigned long greenStart = millis();
while (millis() - greenStart < GREEN_TIME) {
    delay(50);    // 非阻塞等待 (控制單元反覆跳回判斷)
}

// ② 綠燈結束：檢查是否還有行人 (輸入單元 + ALU + 控制單元)
int extendCount = 0;
while (extendCount < MAX_EXTEND && personDetected()) {
    delay(EXTEND_TIME);    // 延長綠燈
    extendCount++;         // ALU：計數器遞增 (SRAM 更新)
}

// ③ 黃燈 → 紅燈 (輸出單元：PORTx 切換)
setLight(false, true, false);    // 黃燈
delay(YELLOW_TIME);
setLight(false, false, true);    // 紅燈
delay(RED_TIME);
```

範例三

超音波防盜警報系統

偵測入侵 → 亮燈 + 蜂鳴器警報 → 按鈕解除

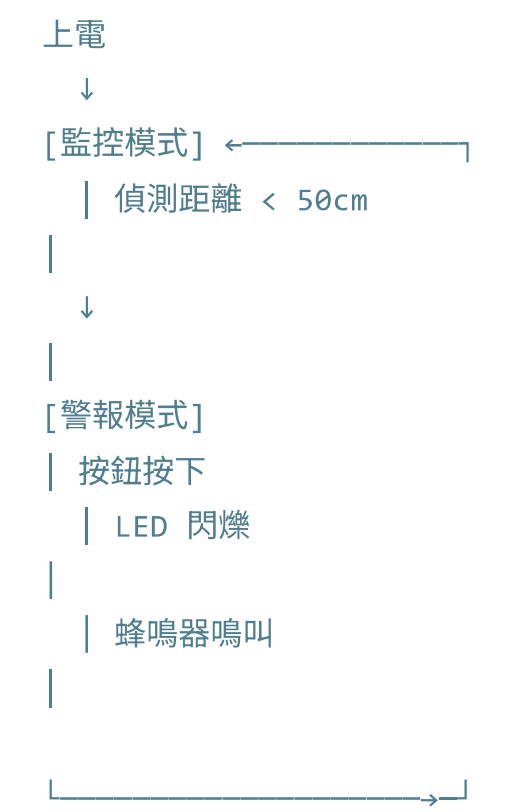
狀態機設計

INPUT_PULLUP

去彈跳 Debounce

防盜警報：功能設計

狀態機設計 (State Machine)



接線說明

元件	Mega 2560
HC-SR04 TRIG	D10
HC-SR04 ECHO	D11
LED	D6 (串 220Ω)

INPUT_PULLUP 原理

- ATmega2560 內建上拉電阻 (20~50kΩ)
- 未按下 → 腳位被拉至 HIGH (5V)
- 按下後 → 腳位接 GND → 讀到 LOW
- 不需外接電阻，電路更簡潔

```
// 設定內建上拉 (DDRx=0 + PORTx=1)
pinMode(BTN_PIN, INPUT_PULLUP);

// 按下時讀到 LOW (反邏輯)
if (digitalRead(BTN_PIN) == LOW) {
    // 按鈕被按下
}
```

💡 按鈕只需一端接 D2，另一端接 GND 即可，不需要額外接電阻。

無源蜂鳴器

D8

解除按鈕D2 → GND

防盜警報：去彈跳與程式碼

按鈕去彈跳 (Debounce)

- 機械按鈕按下瞬間，接點抖動產生多次高低電位變化
- 不處理會造成誤觸發（一按多次）
- 軟體去彈跳：偵測到 LOW 後等待 50ms 再確認

```
if (digitalRead(BTN_PIN)
== LOW) {
    delay(50); // 等彈跳結
    束
    if
(digitalRead(BTN_PIN) ==
LOW) {
        // 確認真的按下了
        stopAlarm();
    }
}
```

警報模式輸出

```
if (alarmOn) {
    // LED 亮 (PORTx 寫 1)
    digitalWrite(LED_PIN,
HIGH);
    // 蜂鳴器持續鳴叫
    (Timer2 PWM)
    tone(BUZZER_PIN,
1000);
    delay(200);

    // LED 閃爍熄滅（蜂鳴器
    不停）
    digitalWrite(LED_PIN,
LOW);
    // 注意：不呼叫
    noTone() !
    delay(100);
}
```

🔑 alarmOn 為 SRAM 中的狀態旗標，控制單元依此決定進入哪個執行區段，這是狀態機設計的核心概念。

範例四

超音波倒車雷達模擬

越近嗶越快 × LED 亮度指示 × 持續長鳴

map() 線性換算

analogWrite PWM

tone() 持續時間

核心概念：map() 線性比例換算

map() 函式語法

```
map(value, fromLow,
    fromHigh, toLow, toHigh)
```

將 value 從範圍 [fromLow ~ fromHigh]

線性映射到 [toLow ~ toHigh]

本程式應用：距離 → 間隔

```
// 距離越近 → 間隔越短 → 嗶越快
int interval = map(dist,
    10,    // 距離最近 (公分)
    200,   // 距離最遠 (公分)
    80,    // 對應最短間隔 (毫秒)
    1000   // 對應最長間隔 (毫秒)
);
```

對照表 (距離 → 間隔)

距離	間隔	感受
200 cm	1000 ms	每秒 1 次 (慢)
150 cm	~ 791 ms	
100 cm	~ 582 ms	每秒約 2 次
50 cm	~ 372 ms	
20 cm	~ 133 ms	每秒約 7 次 (快)
≤ 10 cm	持續	⚠ 持續長鳴

💡 map() 底層就是一條一次方程式：
$$\text{result} = (\text{value} - \text{fromLow}) \times (\text{toHigh} - \text{toLow}) \div (\text{fromHigh} - \text{fromLow}) + \text{toLow}$$

analogWrite() 與 PWM 亮度控制

LED 亮度隨距離變化

```
// 距離越近 → 亮度越高
int brightness =
map(dist,
    CONT_DIST, // 10cm
    → 亮度 255 (全亮)
    MAX_DIST,  // 200cm
    → 亮度 0 (熄滅)
    255, 0
);
analogWrite(LED_PIN,
brightness);
```

constrain() 防止溢位

```
// 確保亮度值在 0~255 之間
brightness =
constrain(brightness, 0,
255);
```

PWM 原理 (Timer4 暫存器)

- Mega D6 腳由 Timer4 的 OC4A 通道控制
- 設定 OCR4A 比較暫存器決定 duty cycle
- $\text{duty cycle} = \text{brightness} \div 255 \times 100\%$
- 頻率約 490Hz (人眼無法感知閃爍)
- 視覺上感知到亮度連續變化

analogWrite 值	duty cycle	LED 亮度
0	0%	熄滅
64	25%	微亮
128	50%	半亮
255	100%	全亮

倒車雷達：嗶聲設計與程式流程

tone() 三參數版本

```
// tone(腳位, 頻率, 持續時間)
// 呼叫後立即返回 (非阻塞)
// Timer2 背景自動停止
tone(BUZZER_PIN, 2000, 60);
delay(interval);
// ↑ 整個週期 = interval ms
// 嗶聲：前 60ms 有聲
// 靜音：後 (interval-60) ms
```

pulseIn() 逾時設定

```
// 第三參數：逾時微秒數
// 30000μs = 30ms ≈ 最遠 500cm
pulseIn(ECHO_PIN, HIGH, 30000);
// 逾時回傳 0 → 視為無障礙物
```

 真實倒車雷達設計：通常有 4~8 個超音波感測器，分佈在車尾偵測不同角度，並有 LED 儀表板顯示距離分段。本實習模擬單點偵測的核心邏輯。

距離區間邏輯

```
if (dist <= 0 || dist > 200) {
    // 範圍外：靜音 + LED 熄滅
    noTone(BUZZER_PIN);
    analogWrite(LED_PIN, 0);
} else if (dist <= 10) {
    // 極近：持續長鳴 (高頻 3000Hz)
    tone(BUZZER_PIN, 3000);
} else {
    // 一般範圍：間歇嗶聲
```

```
    int interval =  
    map(dist, ...);  
    tone(BUZZER_PIN, 2000,  
60);  
    delay(interval);  
}
```

四個範例綜合比較

項目	範例一 基礎警報	範例二 行人號誌	範例三 防盜警報	範例四 倒車雷達
輸入	HC-SR04	HC-SR04	HC-SR04 + 按鈕	HC-SR04
輸出	LED + 蜂鳴器	三色 LED	LED + 蜂鳴器	LED(PWM) + 蜂鳴器
計時方式	delay()	millis()	delay()	delay() + tone 持續時間
狀態管理	無	號誌循環 + 延長計數	狀態機	距離區間判斷
進階函式	—	millis()	INPUT_PULLUP 去彈跳	map() / analogWrite()
蜂鳴器	閃10次後停	無	持續至解除	距離越近越快

本節課重點回顧

硬體

- HC-SR04 : TRIG 觸發、ECHO 回波
- 距離 = 時間 $\times 0.034 \div 2$
- 無源蜂鳴器用 `tone()` 驅動

軟體 / 微電腦觀念

- 五大單元在程式中的對應
- DDRx / PORTx / PINx 暫存器
- `pulseIn()` 使用 Timer1 計時
- `tone()` 使用 Timer2 產生 PWM

檔案位置：114學年2微算機實習 / 超音波感測 / 超音波感測.ino